

MLOps

AWS Lambda – IBM watsonx.ai – Evidently – IBM watsonx.governance

Session 7P

Juan Ramírez
jramirezg@faculty.ie.edu

AWS Lambda

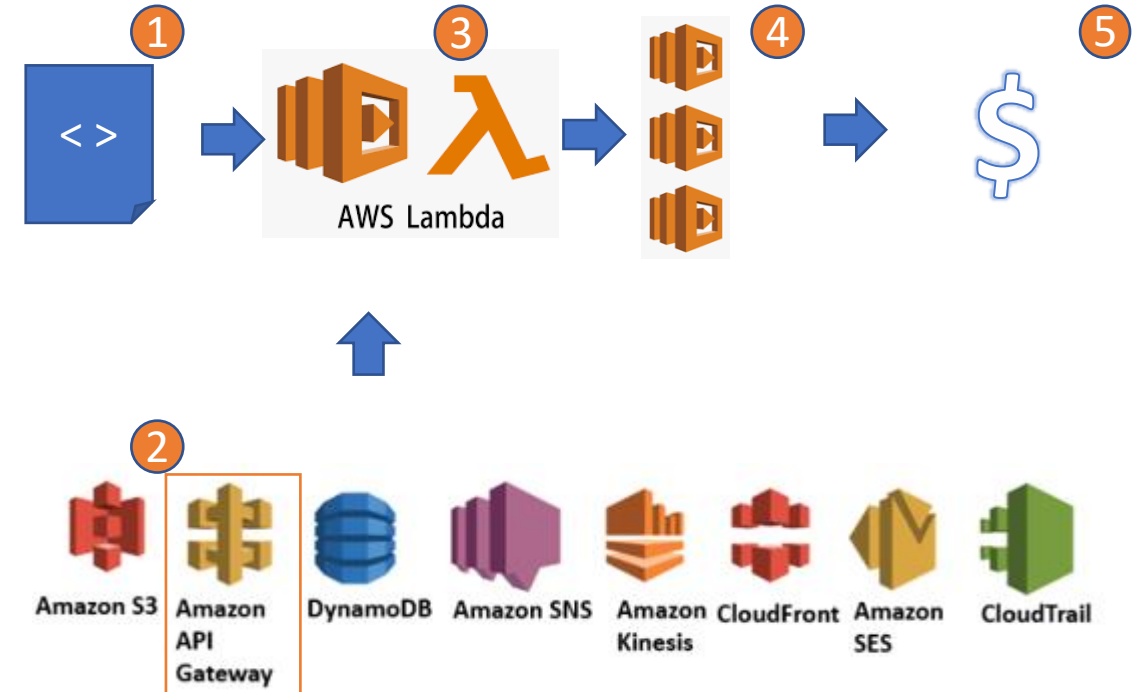
What is AWS Lambda?

<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

- AWS Lambda is a **compute service** that lets you run code **without provisioning or managing servers**
- Lambda runs your code on a high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, and logging
- You organize your code into **Lambda functions**. Lambda runs your function only when needed and scales automatically, from a few requests per day to thousands per second
- You pay only for the compute time that you consume—there is no charge when your code is not running

How does AWS Lambda work?

1. Upload your AWS Lambda code in any language supported by AWS Lambda (Java, Python, Go, and C# are some of the languages supported by AWS Lambda function)
2. These are some AWS services which allow you to trigger AWS Lambda
3. AWS Lambda helps you to upload code and the event details on which it should be triggered
4. Executes AWS Lambda Code when it is triggered by AWS services
5. AWS charges only when the AWS lambda code executes, and not otherwise.



Steps

1. Build a Docker Image that will generate prediction using a trained model.
2. Creating a repository on AWS ECR
3. Pushing the docker image to AWS ECR
4. Create the lambda function using the container image
5. Using the API gateway to access the prediction API.

<https://betterprogramming.pub/deploying-ml-models-using-aws-lambda-and-api-gateway-f1f349515c81>

1. Build a Docker Image

- Create Lambda Function

```
lambda_function.py > ...
1 import pickle
2
3 filename = 'model.pkl'
4 model = pickle.load(open(filename, 'rb'))
5
6 def predict(features):
7     return model.predict(features).tolist()
8
9 def lambda_handler(event, context):
10     values = event['values']
11     result = predict(values)
12     return result
```

- Create Dockerfile

```
Dockerfile > ...
1 ## downloading lambda image
2 FROM public.ecr.aws/lambda/python:3.8
3 RUN pip install scikit-learn
4 COPY model.pkl .
5 COPY lambda_function.py .
6 CMD [ "lambda_function.lambda_handler" ]
```

→ Lambda function needs a `lambda_handler` that is the method in the function that 'handle' or processes events

- Build Docker image and make the container to test the image locally
`docker build -t iris_image .`
`docker run -it --rm -p 5000:8080 iris_image:latest`
- We can test the model trying this url that AWS provides
`http://localhost:5000/2015-03-31/functions/function/invocations`

2. Creating a repository on Amazon ECR

What is Amazon ECR?

Amazon ECR is a fully managed container registry offering high-performance hosting, so you can reliably deploy application images and artifacts anywhere.



Install AWS CLI and execute:

```
aws ecr create-repository --repository-name iris-registry
```

```
c:\JM\Python_Projects\07_01 AWS Lambda>aws ecr create-repository --repository-name iris-registry
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:eu-west-3:916669174304:repository/iris-registry",
    "registryId": "916669174304",
    "repositoryName": "iris-registry",
    "repositoryUri": "916669174304.dkr.ecr.eu-west-3.amazonaws.com/iris-registry",
    "createdAt": "2023-01-22T13:01:06+01:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

Note down this info for later

3. Push the docker image to AWS ECR

<https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-push-ecr-image.html>

1. Get docker authentication to the repository where you want to push the image

```
c:\JM\Python_Projects\07_01 AWS Lambda>aws ecr get-login-password --region eu-west-3 | docker login --username AWS --password stdin 916669174304.dkr.ecr.eu-west-3.amazonaws.com
Login Succeeded
```

2. Tag the image

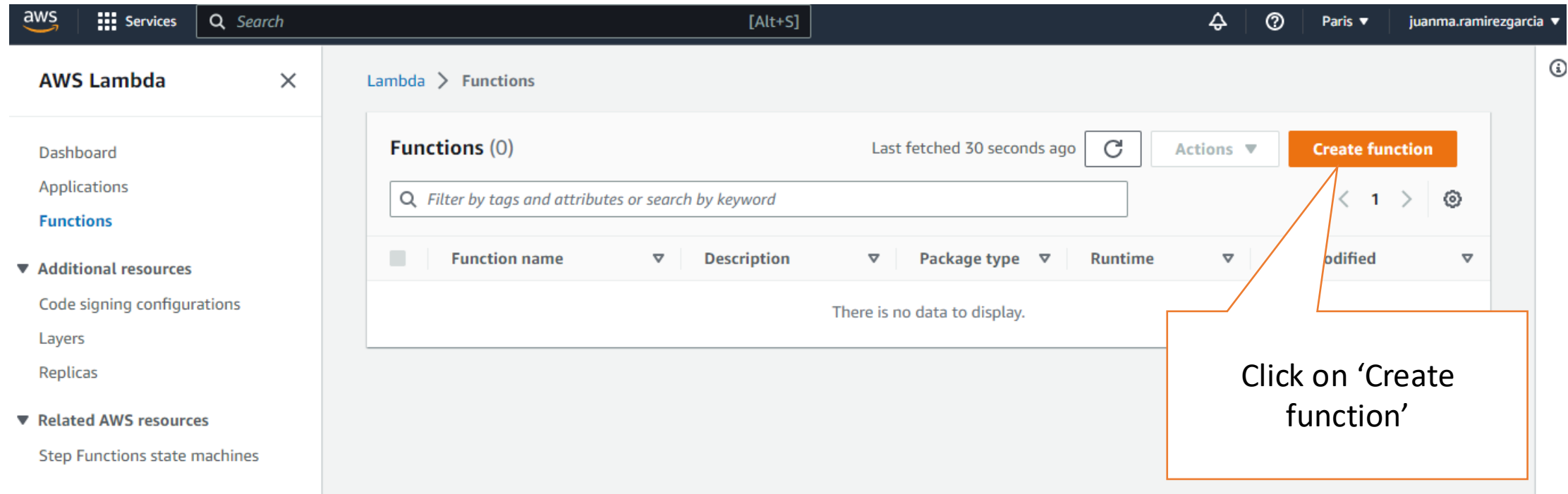
```
c:\JM\Python_Projects\07_01 AWS Lambda>docker tag a1bf9d49b891 916669174304.dkr.ecr.eu-west-3.amazonaws.com/iris-registry:tag
```

3. Push the image

```
c:\JM\Python_Projects\07_01 AWS Lambda>docker push 916669174304.dkr.ecr.eu-west-3.amazonaws.com/iris-registry:tag
The push refers to repository [916669174304.dkr.ecr.eu-west-3.amazonaws.com/iris-registry]
00dc64cf877e: Pushed
aac9ad9598ac: Pushed
dbcda9f908767: Pushed
7b29a54d4abf: Pushed
3837a9c028ca: Pushed
2d244e0816c6: Pushed
13375038cb04: Pushed
c9799687f2b3: Pushed
d30cec5cc7ec: Pushed
tag: digest: sha256:9e6a0c581b673d3c1a6b00bf8064d0de2be5016e9fc49a3d23d5054383f00498 size: 2210
```

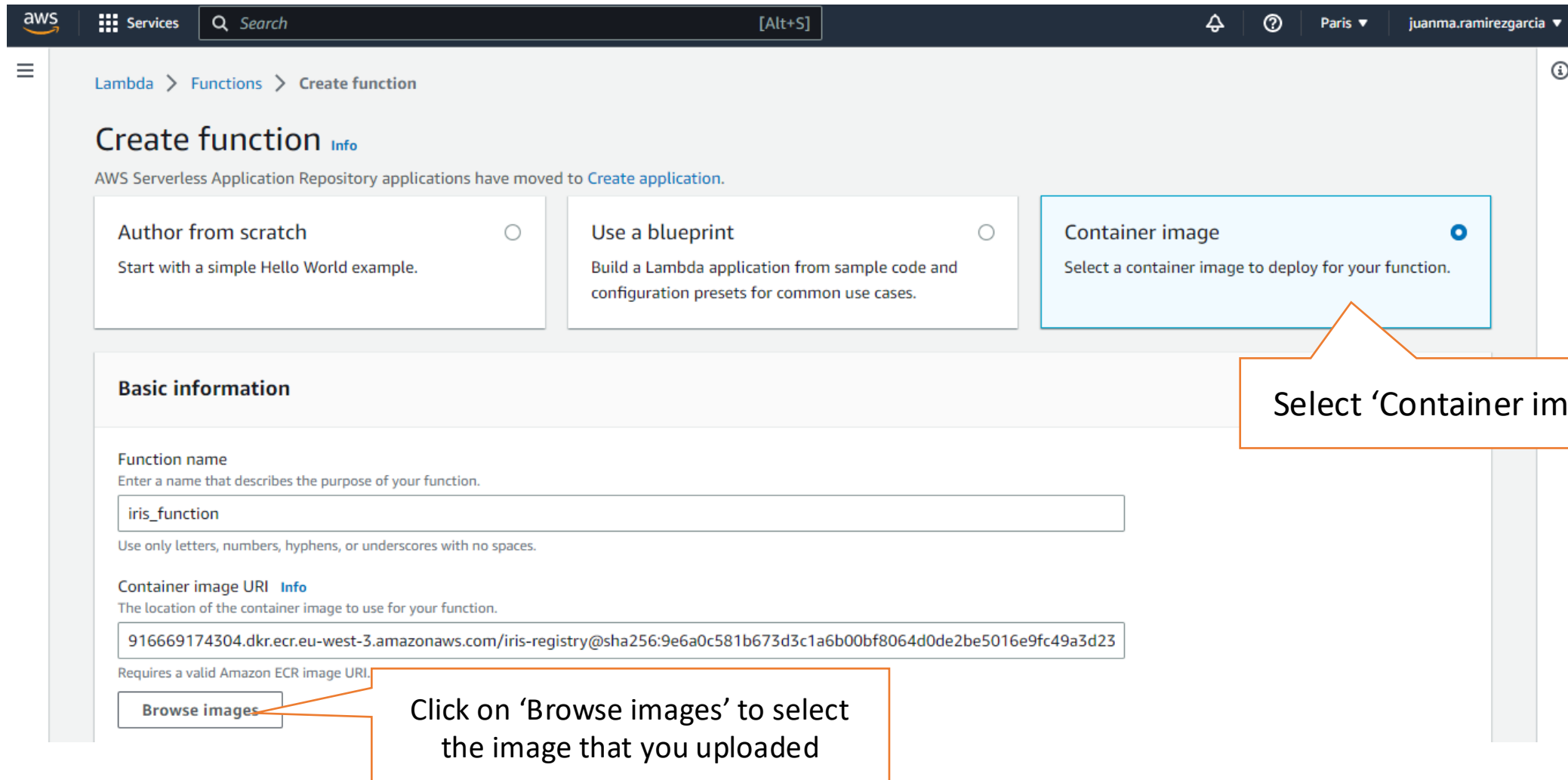
4. Create Lambda Function Using Container Image

- Go to AWS Lambda in your AWS account



4. Create Lambda Function Using Container Image

- Go to AWS Lambda in your AWS account



aws Services Search [Alt+S] Paris juanma.ramirezgarcia

Lambda > Functions > Create function

Create function [Info](#)

AWS Serverless Application Repository applications have moved to [Create application](#).

Author from scratch
Start with a simple Hello World example.

Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.

Container image
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Container image URI [Info](#)
The location of the container image to use for your function.

Requires a valid Amazon ECR image URI.

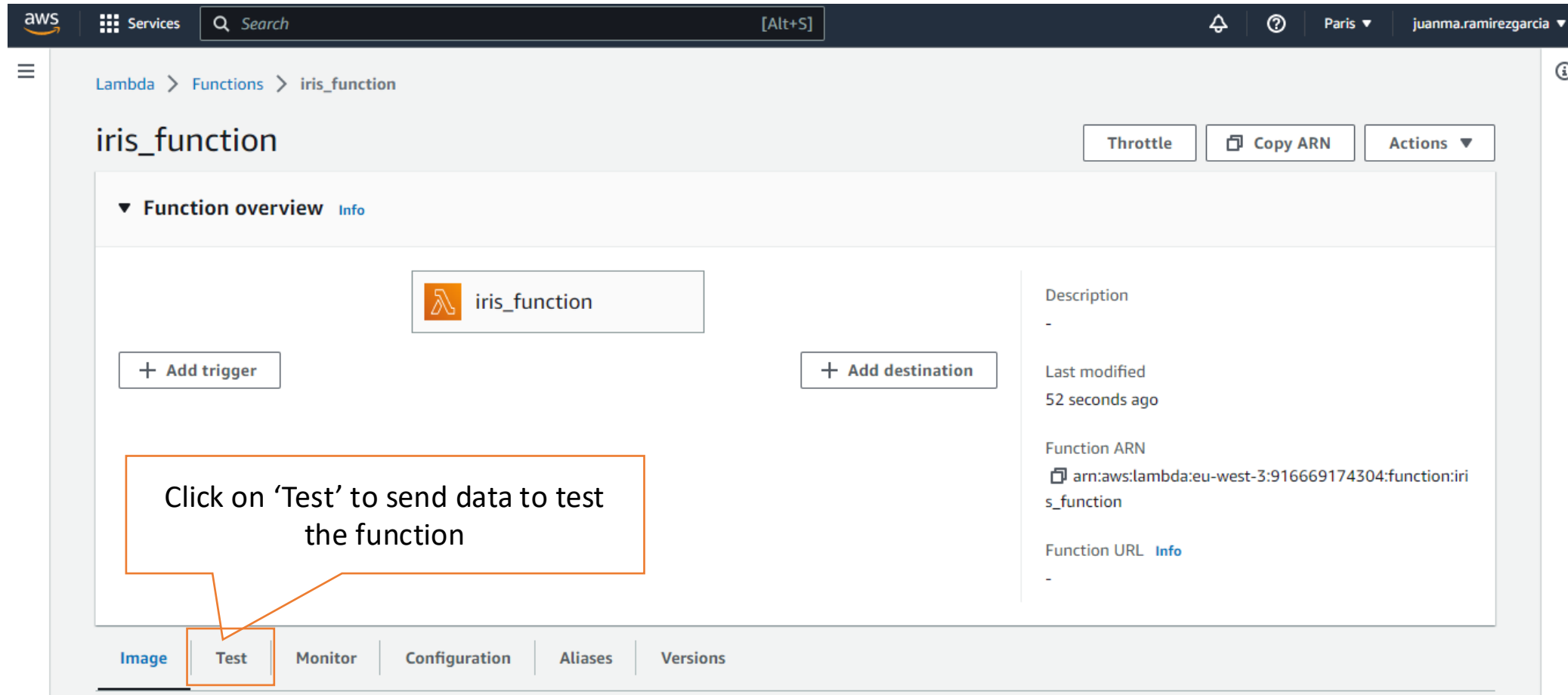
[Browse images](#)

Select 'Container image'

Click on 'Browse images' to select the image that you uploaded

4. Create Lambda Function Using Container Image

- Go to AWS Lambda in your AWS account



4. Create Lambda Function Using Container Image

- Test function

aws Services Search [Alt+S] Paris juanma.ramirezgarcia

Image **Test** Monitor Configuration Aliases Versions

Test event Info Save Test

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Event name

iris_test

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same

Template - optional

hello-world

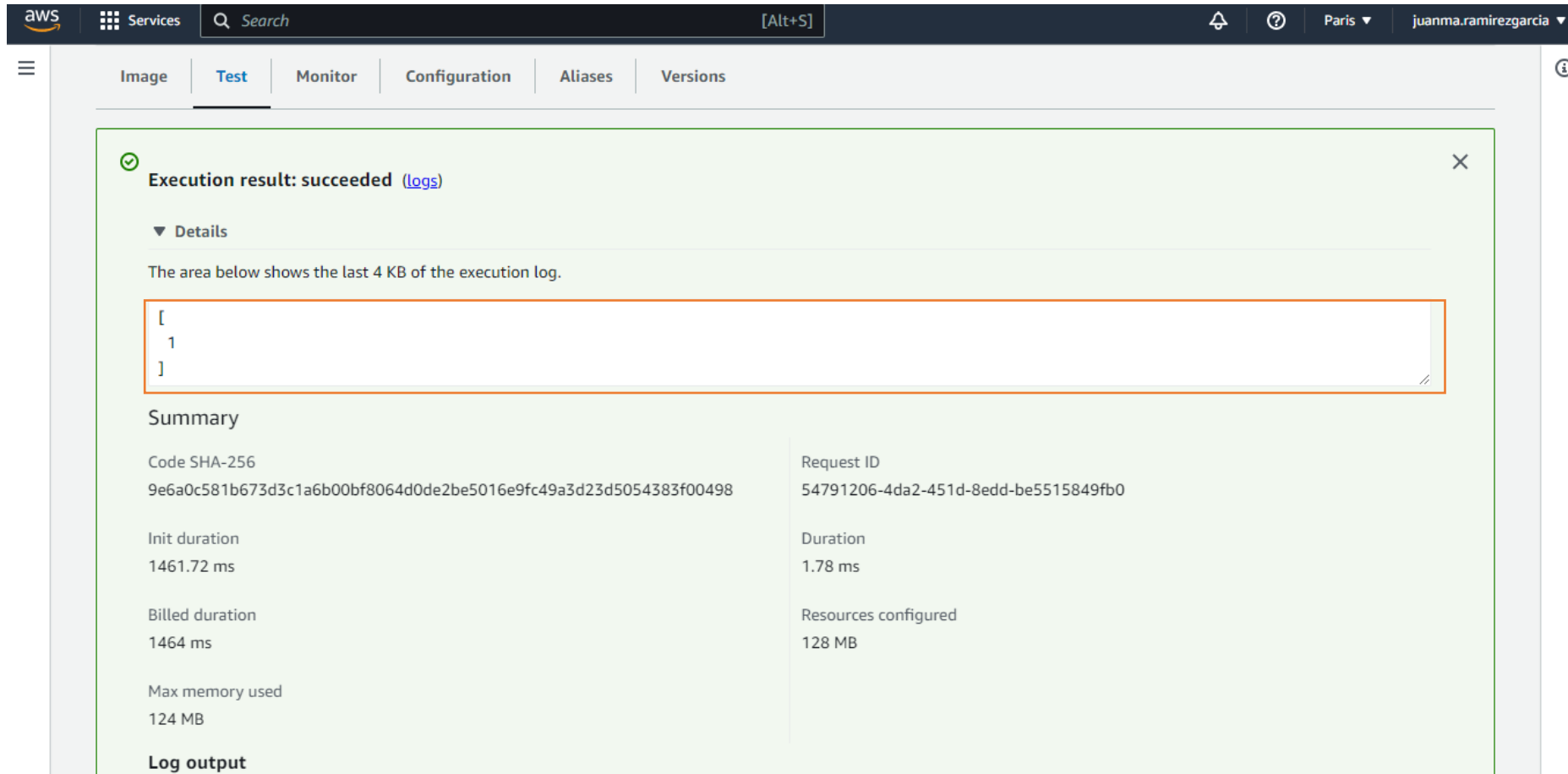
Event JSON Format JSON

1 [{"rvalues": [[0.1, 2, 0.1, 3]]}]

Create a payload to test

4. Create Lambda Function Using Container Image

- Test result



The screenshot displays the AWS Lambda console interface. At the top, the navigation bar includes the AWS logo, a 'Services' menu, a search bar, and user information for 'juanma.ramirezgarcia'. The main content area has tabs for 'Image', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Test' tab is selected, showing a green banner with a checkmark icon and the text 'Execution result: succeeded (logs)'. Below this, a 'Details' section indicates that the following area shows the last 4 KB of the execution log. A text box contains the log output:

```
[  
  1  
]
```

. A 'Summary' section is divided into two columns. The left column lists: 'Code SHA-256' (9e6a0c581b673d3c1a6b00bf8064d0de2be5016e9fc49a3d23d5054383f00498), 'Init duration' (1461.72 ms), 'Billed duration' (1464 ms), and 'Max memory used' (124 MB). The right column lists: 'Request ID' (54791206-4da2-451d-8edd-be5515849fb0), 'Duration' (1.78 ms), and 'Resources configured' (128 MB). At the bottom of the summary is a 'Log output' section.

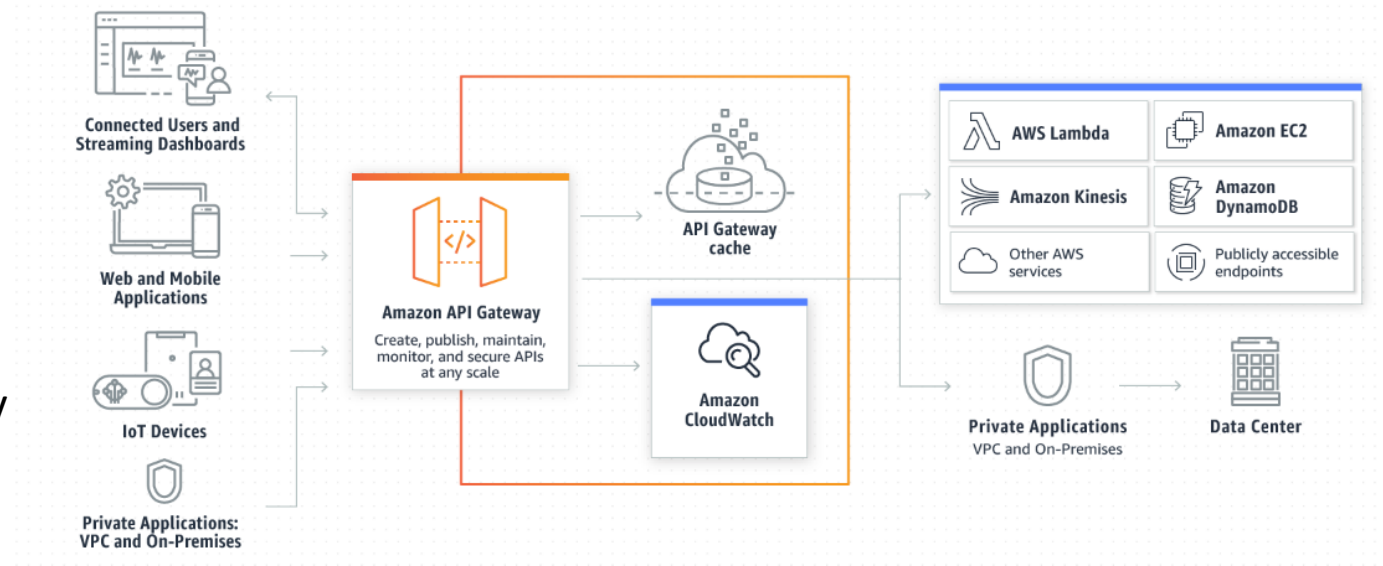
Summary	
Code SHA-256	Request ID
9e6a0c581b673d3c1a6b00bf8064d0de2be5016e9fc49a3d23d5054383f00498	54791206-4da2-451d-8edd-be5515849fb0
Init duration	Duration
1461.72 ms	1.78 ms
Billed duration	Resources configured
1464 ms	128 MB
Max memory used	
124 MB	

5. Using the API gateway to access the prediction API

What is Amazon API Gateway

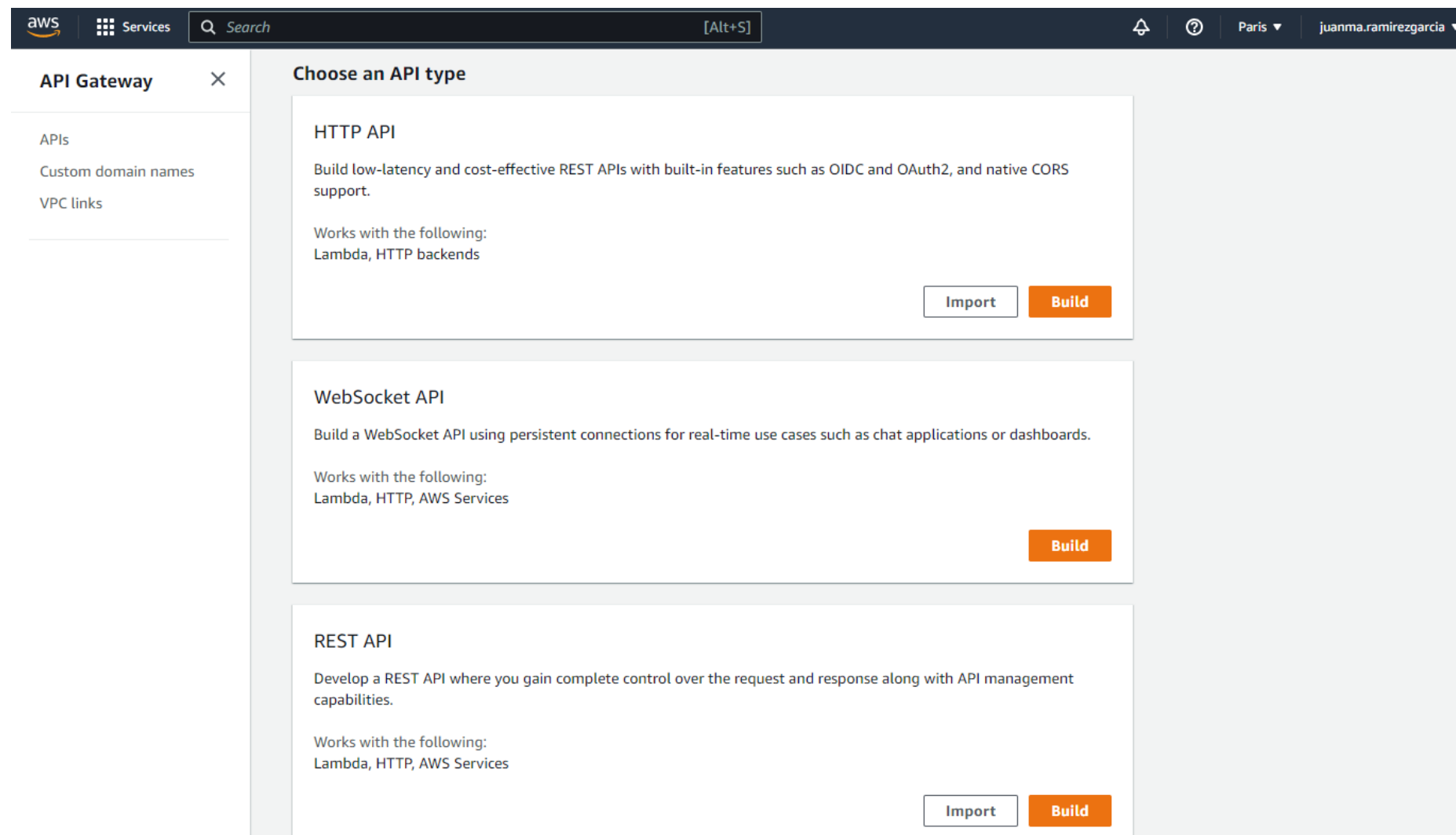
- Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale.
- APIs act as the "front door" for applications to access data, business logic, or functionality from your backend services.
- Using API Gateway, you can create RESTful APIs and WebSocket APIs that enable real-time two-way communication applications.
- API Gateway supports containerized and serverless workloads, as well as web applications.

<https://aws.amazon.com/api-gateway/>



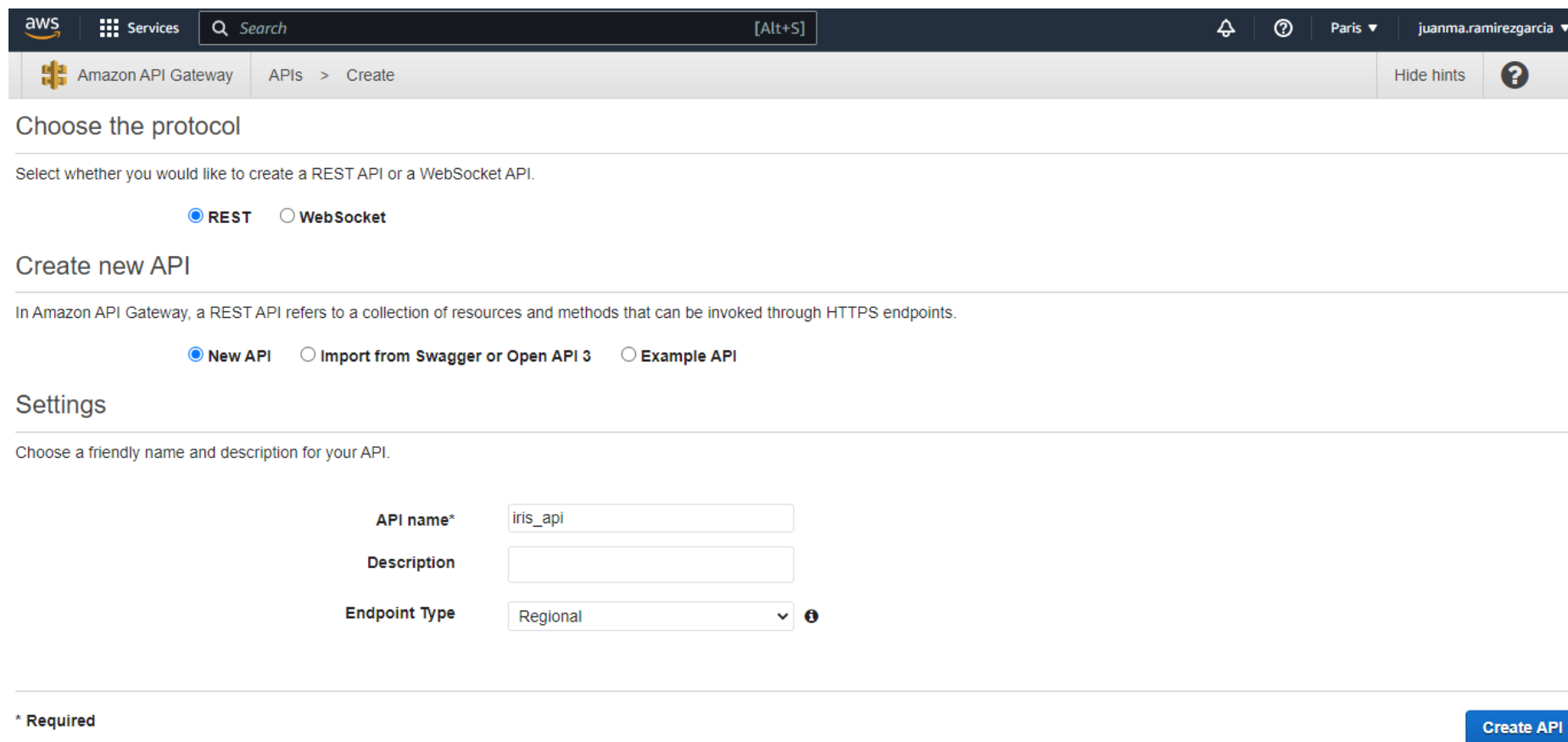
5. Using the API gateway to access the prediction API

Go to API Gateway on your AWS account and select Build REST API



5. Using the API gateway to access the prediction API

Select REST – New API – and give an API name



The screenshot shows the AWS Management Console interface for creating a new API. The top navigation bar includes the AWS logo, 'Services' menu, a search bar, and user information. The breadcrumb trail indicates the path: Amazon API Gateway > APIs > Create. The main content area is titled 'Choose the protocol' and instructs the user to select between REST and WebSocket. The 'REST' option is selected. Below this, the 'Create new API' section is shown, with instructions and three options: 'New API' (selected), 'Import from Swagger or Open API 3', and 'Example API'. The 'Settings' section follows, asking for a friendly name and description. The 'API name*' field contains 'iris_api', the 'Description' field is empty, and the 'Endpoint Type' dropdown is set to 'Regional'. A '* Required' note is at the bottom left, and a 'Create API' button is at the bottom right.

aws Services Search [Alt+S]

Amazon API Gateway APIs > Create

Hide hints ?

Choose the protocol

Select whether you would like to create a REST API or a WebSocket API.

☒ REST ☐ WebSocket

Create new API

In Amazon API Gateway, a REST API refers to a collection of resources and methods that can be invoked through HTTPS endpoints.

☒ New API ☐ Import from Swagger or Open API 3 ☐ Example API

Settings

Choose a friendly name and description for your API.

API name*

Description

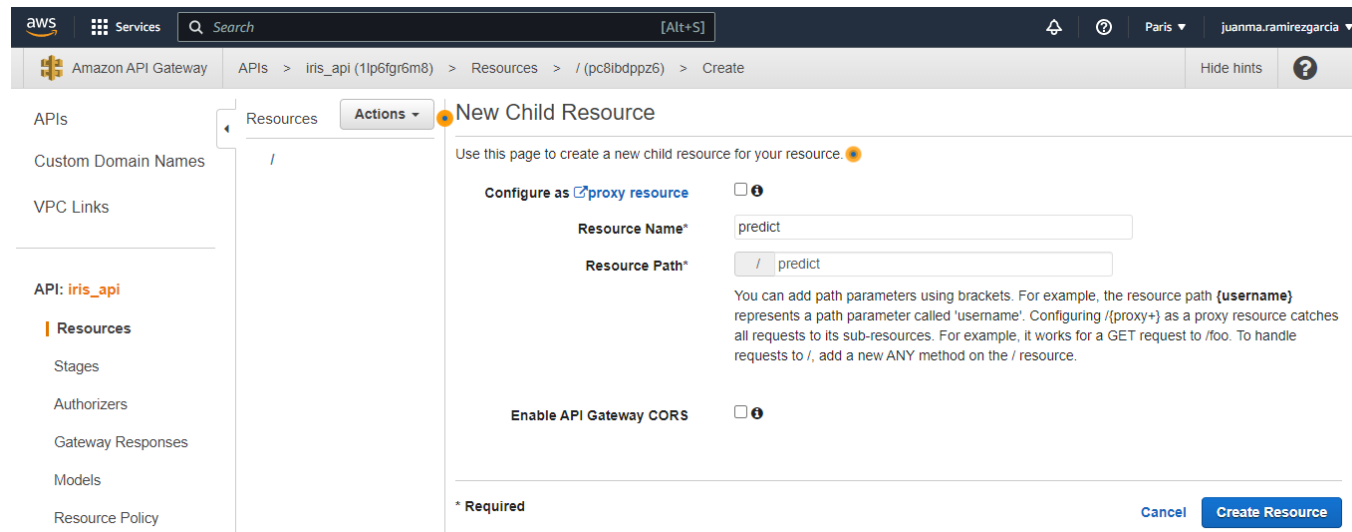
Endpoint Type ⓘ

* Required

Create API

5. Using the API gateway to access the prediction API

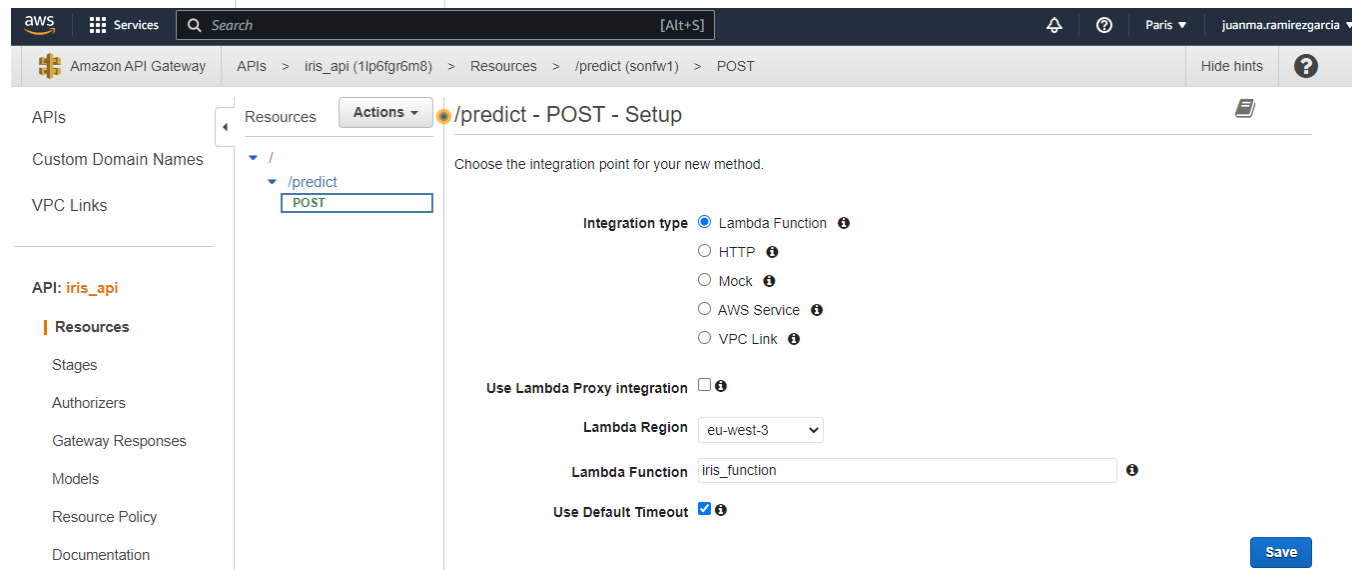
Create a new resource and then a new method



The screenshot shows the AWS API Gateway console. The breadcrumb trail is: Amazon API Gateway > APIs > iris_api (1lp6fgr6m8) > Resources > / (pc8ibdpz6) > Create. The left sidebar shows the API: iris_api with a 'Resources' tab selected. The main panel is titled 'New Child Resource' and contains the following fields:

- Configure as**: ☒ proxy resource
- Resource Name***: predict
- Resource Path***: / predict
- Enable API Gateway CORS**: ☐

Below these fields is a note: "You can add path parameters using brackets. For example, the resource path {username} represents a path parameter called 'username'. Configuring /{proxy+} as a proxy resource catches all requests to its sub-resources. For example, it works for a GET request to /foo. To handle requests to /, add a new ANY method on the / resource." At the bottom right are 'Cancel' and 'Create Resource' buttons.



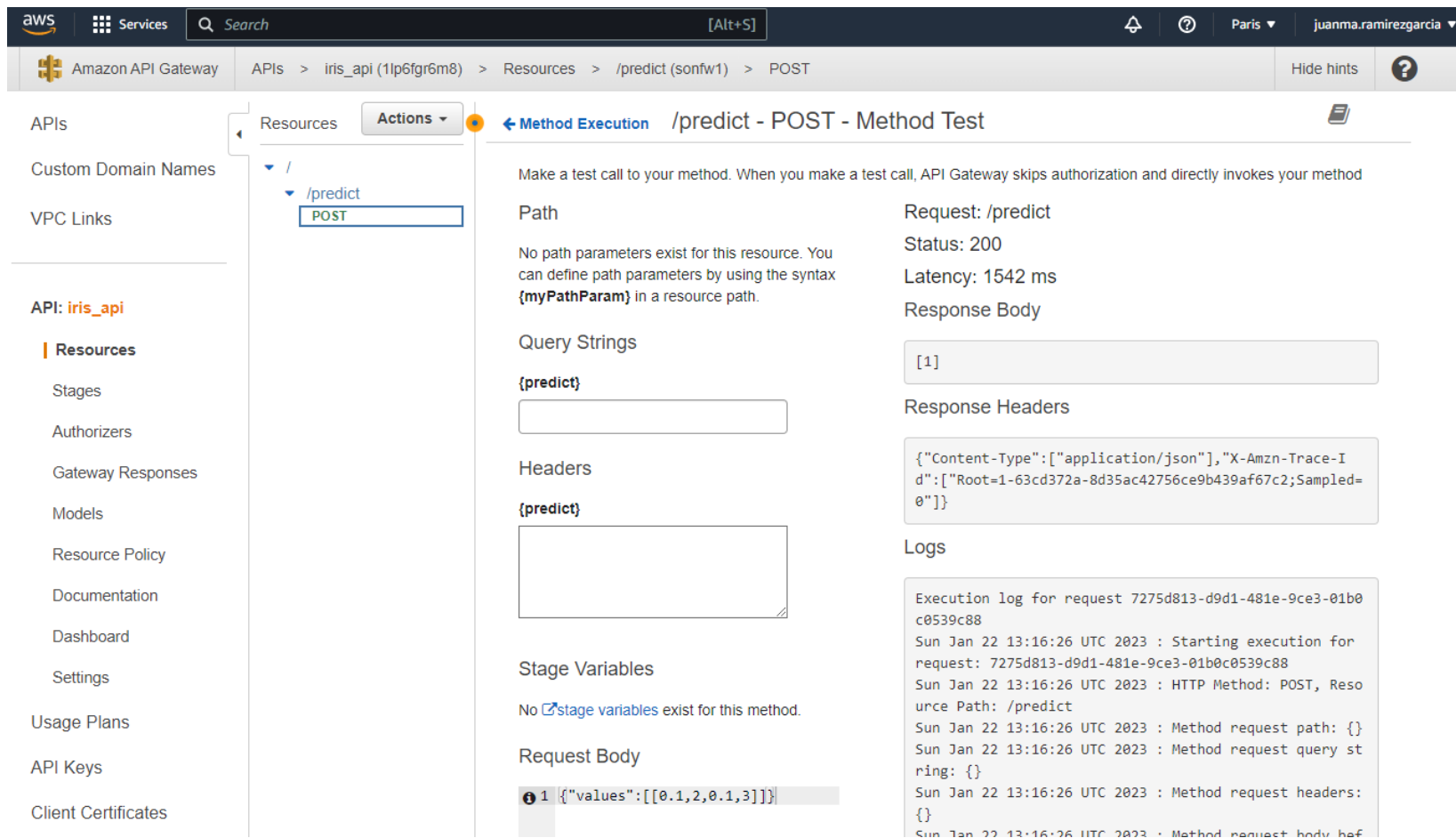
The screenshot shows the AWS API Gateway console. The breadcrumb trail is: Amazon API Gateway > APIs > iris_api (1lp6fgr6m8) > Resources > /predict (sonfw1) > POST. The left sidebar shows the API: iris_api with a 'Resources' tab selected. The main panel is titled '/predict - POST - Setup' and contains the following fields:

- Integration type**: ☒ Lambda Function, ☐ HTTP, ☐ Mock, ☐ AWS Service, ☐ VPC Link
- Use Lambda Proxy integration**: ☐
- Lambda Region**: eu-west-3
- Lambda Function**: iris_function
- Use Default Timeout**: ☒

At the bottom right is a 'Save' button.

5. Using the API gateway to access the prediction API

Test the method



The screenshot shows the AWS API Gateway console interface. The breadcrumb navigation at the top indicates the path: Amazon API Gateway > APIs > iris_api (1lp6fgr6m8) > Resources > /predict (sonfw1) > POST. The left sidebar contains navigation links for APIs, Custom Domain Names, VPC Links, and a list of API resources including iris_api, Resources, Stages, Authorizers, Gateway Responses, Models, Resource Policy, Documentation, Dashboard, Settings, Usage Plans, API Keys, and Client Certificates. The main content area is titled '/predict - POST - Method Test' and includes a description: 'Make a test call to your method. When you make a test call, API Gateway skips authorization and directly invokes your method'. It features several input fields for Path, Query Strings, Headers, and Stage Variables, all currently empty. The Request Body field contains a JSON object: `{ "values": [[0.1, 2, 0.1, 3]] }`. On the right, the test results are displayed: Request: /predict, Status: 200, Latency: 1542 ms, and Response Body: `[1]`. Below these, the Response Headers are shown as a JSON object: `{ "Content-Type": ["application/json"], "X-Amzn-Trace-ID": ["Root=1-63cd372a-8d35ac42756ce9b439af67c2;Sampled=0"] }`. The Logs section at the bottom provides an execution log for the request, detailing the start time, method (POST), path, query string, headers, and body.

APIs

Custom Domain Names

VPC Links

API: **iris_api**

Resources

Stages

Authorizers

Gateway Responses

Models

Resource Policy

Documentation

Dashboard

Settings

Usage Plans

API Keys

Client Certificates

Resources

Actions

Method Execution /predict - POST - Method Test

Make a test call to your method. When you make a test call, API Gateway skips authorization and directly invokes your method

Path

No path parameters exist for this resource. You can define path parameters by using the syntax `{myPathParam}` in a resource path.

Query Strings

`{predict}`

Headers

`{predict}`

Stage Variables

No [stage variables](#) exist for this method.

Request Body

`{ "values": [[0.1, 2, 0.1, 3]] }`

Request: /predict

Status: 200

Latency: 1542 ms

Response Body

`[1]`

Response Headers

`{ "Content-Type": ["application/json"], "X-Amzn-Trace-ID": ["Root=1-63cd372a-8d35ac42756ce9b439af67c2;Sampled=0"] }`

Logs

Execution log for request 7275d813-d9d1-481e-9ce3-01b0c0539c88

Sun Jan 22 13:16:26 UTC 2023 : Starting execution for request: 7275d813-d9d1-481e-9ce3-01b0c0539c88

Sun Jan 22 13:16:26 UTC 2023 : HTTP Method: POST, Resource Path: /predict

Sun Jan 22 13:16:26 UTC 2023 : Method request path: {}

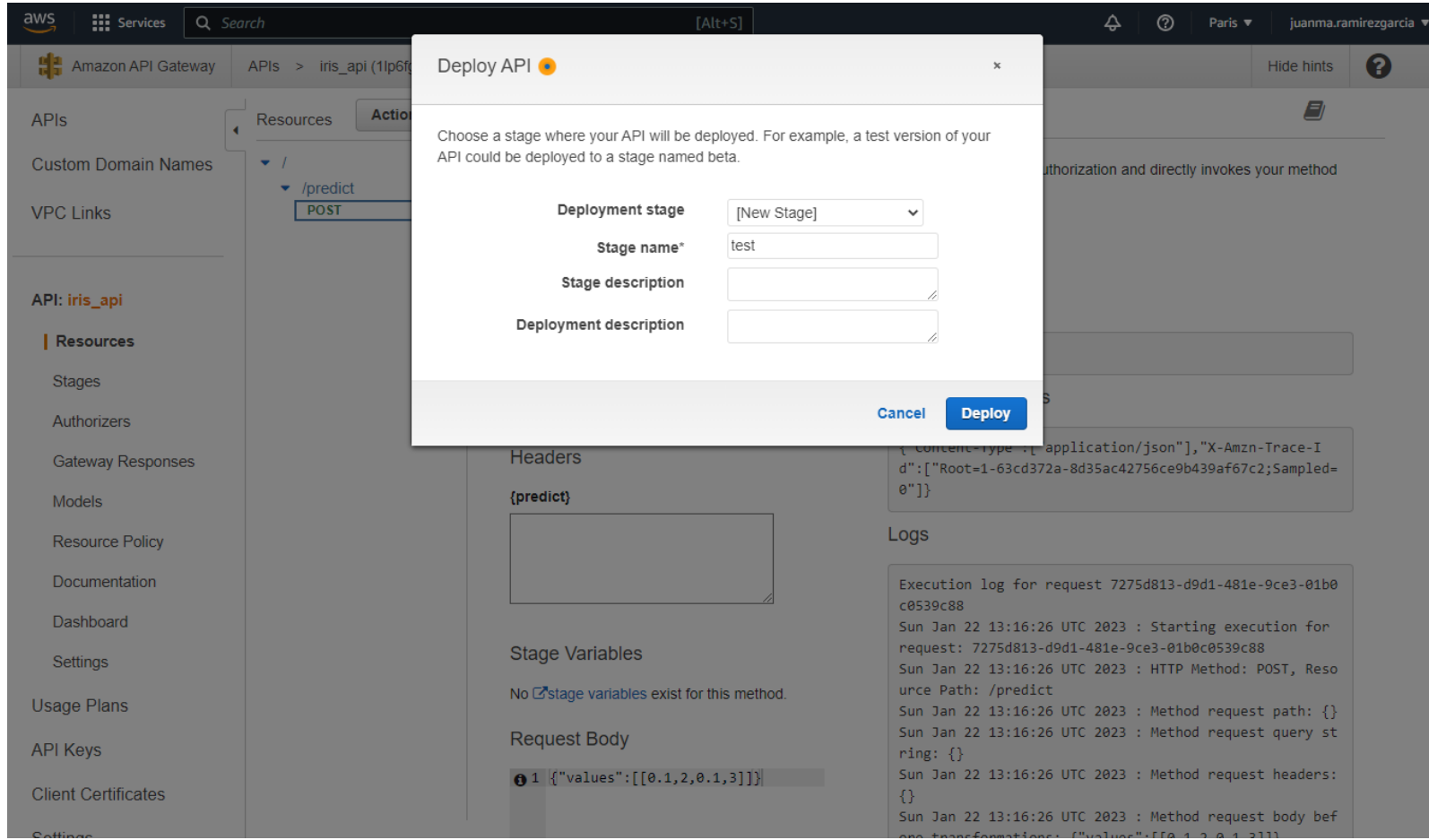
Sun Jan 22 13:16:26 UTC 2023 : Method request query string: {}

Sun Jan 22 13:16:26 UTC 2023 : Method request headers: {}

Sun Jan 22 13:16:26 UTC 2023 : Method request body: {}

5. Using the API gateway to access the prediction API

Deploy the API



5. Using the API gateway to access the prediction API

Now we can test with 'Invoke URL'

Be careful because the url need to finish with the resource '/predict':

'https://1lp6fgr6m8.execute-api.eu-west-3.amazonaws.com/test/predict'

The screenshot displays the AWS API Gateway console interface. The top navigation bar includes the AWS logo, 'Services', a search bar, and user information for 'juanma.ramirezgarcia'. The breadcrumb trail indicates the path: 'Amazon API Gateway' > 'APIs' > 'iris_api (1lp6fgr6m8)' > 'Stages' > 'test'. On the left sidebar, the 'APIs' section is expanded, showing 'API: iris_api' and 'Stages' as the active item. The main content area is titled 'test Stage Editor' and features a 'Create' button. Below the title, the 'Invoke URL' is displayed as 'https://1lp6fgr6m8.execute-api.eu-west-3.amazonaws.com/test'. A series of tabs are visible: 'Settings' (selected), 'Logs/Tracing', 'Stage Variables', 'SDK Generation', 'Export', 'Deployment History', and 'Documentation History'. Under the 'Settings' tab, the 'Canary' sub-tab is active. The 'Cache Settings' section shows 'Enable API cache' as an unchecked checkbox. The 'Default Method Throttling' section includes a descriptive paragraph and an 'Enable throttling' checkbox, which is checked. Below this, the 'Rate' is set to '10000 requests per second' and the 'Burst' is set to '5000 requests'.

Steps

- Install AWS CLI <https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-welcome.html>
 - The AWS Command Line Interface (AWS CLI) is an open source tool that enables you to interact with AWS services using commands in your command-line shell
- Install AWS SAM CLI <https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>
 - The AWS Serverless Application Model (AWS SAM) is an open-source framework that you can use to build serverless applications on AWS.
 - A serverless application is a combination of Lambda functions, event sources, and other resources that work together to perform tasks. Note that a serverless application is more than just a Lambda function—it can include additional resources such as APIs, databases, and event source mappings.

watsonx.ai Runtime IBM Cloud

Services Catalog on IBM Cloud

watsonx Assistant By IBM IBM watsonx Assistant lets you build conversational interfaces into any application, device, or channel. Service 	watsonx.ai Studio By IBM (Formerly known as Watson Studio) Develop powerful AI solutions with an integrated collaborative studio and... Service 	Annotator for Clinical Data  By IBM Analyze text to extract medical codes and concepts such as diseases, lab values, medications, procedures and more. Service 	IBM Knowledge Catalog By IBM Discover, profile, catalog and share trusted data in your organization. Service 	IBM Match 360 with Watson By IBM IBM Match 360 with Watson (Match 360) improves trust in AI pipelines by identifying duplicate records and... Service 	IBM watsonx Code Assistant By IBM Leverage tailor-made foundation models for AI-generated code recommendations to accelerate developer productivity. Service 
Knowledge Studio By IBM Teach Watson the language of your domain. Service 	Natural Language Understanding By IBM Analyze text to extract meta-data from content such as concepts, entities, emotion, relations, sentiment and more. Service 	OpenPages By IBM Simplify how you manage risk and regulatory compliance with a unified GRC platform fueled by AI and all your data. Service 	Speech to Text By IBM Low-latency, streaming transcription Service 	Text to Speech By IBM Synthesizes natural-sounding speech from text. Service 	Watson Discovery By IBM Add a cognitive search and content analytics engine to applications. Service 
watsonx By IBM Build generative AI and machine learning solutions. Train, validate, tune, and deploy foundation and machine learning... Service 	watsonx Orchestrate By IBM Lighten your workload with IBM watsonx Orchestrate, powered by LLMs. Streamline workflows and automate tas... Service 	watsonx.ai Runtime By IBM (Formerly known as Watson Machine Learning) Quickly build, run and manage generative AI and machine learning... Service 	watsonx.governance By IBM Accelerate responsibility, transparency and explainability in your data and AI workflows Service 	Watsonx.ai SaaS with Assistant and Governance By IBM Deploy watsonx.ai SaaS to automate the deployment and setup of the IBM watsonx platform. Deployable architecture Terraform IBM Modules (TIM) 	

What is watsonx.ai Runtime

- watsonx.ai Runtime is a service on IBM Cloud with features for training and deploying machine learning models and neural networks.
- Built on a scalable, open source platform based on Kubernetes and Docker components, watsonx.ai Runtime enables you to build, train, deploy, and manage machine learning and deep learning models.

https://dataplatform.cloud.ibm.com/docs/content/wsj/analyze-data/ml-overview_local.html?context=cpdaas&audience=wdp

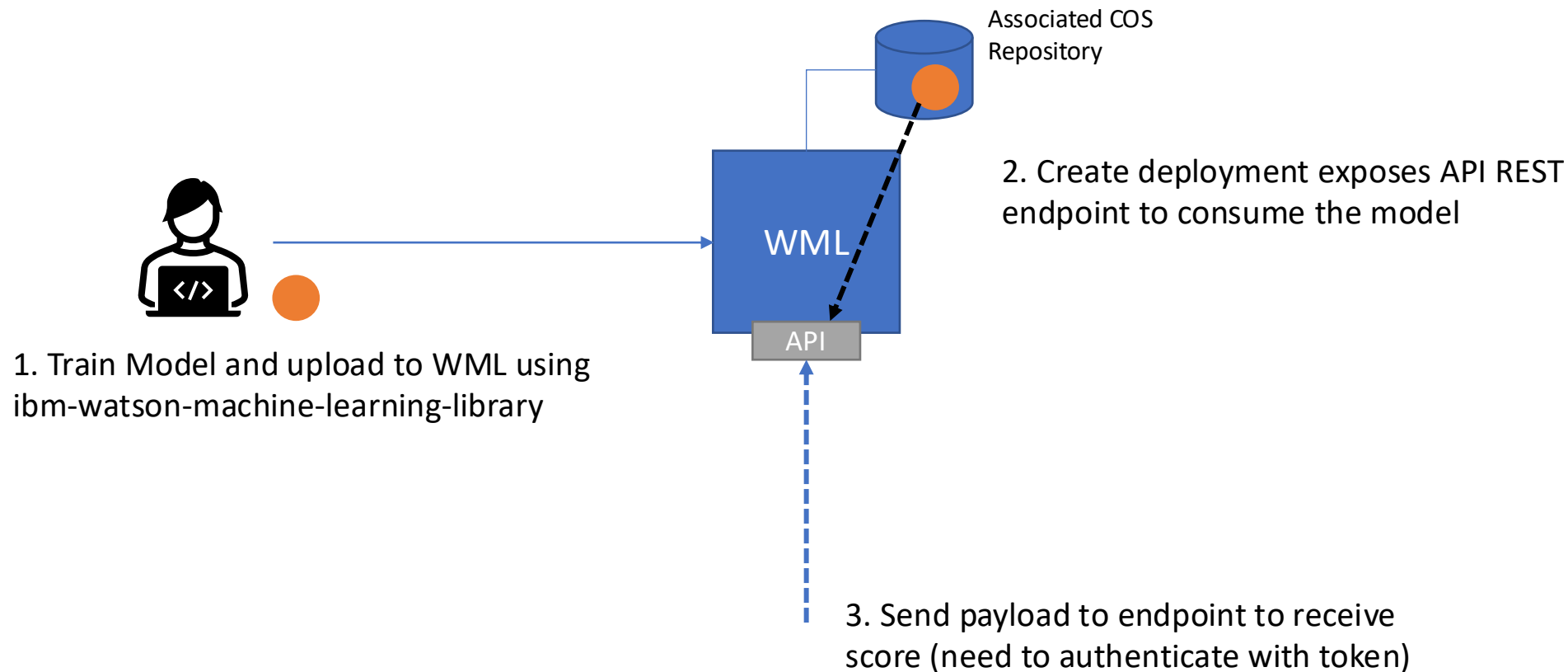
Ibm-watsonx-ai Python Library

- The ibm-watsonx-ai Python library allows you to work with IBM Watson Machine Learning services.
- You can train, store, and deploy your models, score them using APIs, and finally integrate them with your application development.
- The ibm-watsonx-ai library can work with watsonx.ai as a Service or deployed on premises or on any cloud
- Use any supported framework

Framework	Versions	Model Type	Software specification
Decision Optimization	20.1	do-docplex_20.1 do-opl_20.1 do-cplex_20.1 do-cpo_20.1	do_20.1
Decision Optimization	22.1	do-docplex_22.1 do-opl_22.1 do-cplex_22.1 do-cpo_22.1	do_22.1
PMML	3.0 to 4.3	pmml_ (or) pmml_.*3.0 - 4.3	pmml-3.0_4.3
PyTorch	2.0	pytorch-onnx_2.0 pytorch-onnx_rt23.1	runtime-23.1-py3.10 pytorch-onnx_rt23.1-py3.10 pytorch-onnx_rt23.1-py3.10-edt pytorch-onnx_rt23.1-py3.10-dist
PyTorch	2.1	pytorch-onnx_2.1 pytorch-onnx_rt24.1	runtime-24.1-py3.11 pytorch-onnx_rt24.1-py3.11 pytorch-onnx_rt24.1-py3.11-edt pytorch-onnx_rt24.1-py3.11-dist
Python Functions	NA	NA	runtime-24.1-py3.11
Python Scripts	NA	NA	runtime-24.1-py3.11
Scikit-learn	1.1	scikit-learn_1.1	runtime-23.1-py3.10
Scikit-learn	1.3	scikit-learn_1.3	runtime-24.1-py3.11

Framework	Versions	Model Type	Software specification
Spark	3.3	mllib_3.3(deprecated)	spark-mllib_3.3(deprecated)
Spark	3.4	mllib_3.4	spark-mllib_3.4
SPSS	17.1	spss-modeler_17.1	spss-modeler_17.1
SPSS	18.1	spss-modeler_18.1	spss-modeler_18.1
SPSS	18.2	spss-modeler_18.2	spss-modeler_18.2
Tensorflow	2.12	tensorflow_2.12 tensorflow_rt23.1	runtime-23.1-py3.10 tensorflow_rt23.1-py3.10-dist tensorflow_rt23.1-py3.10-edt tensorflow_rt23.1-py3.10
Tensorflow	2.14	tensorflow_2.14 tensorflow_rt24.1	runtime-24.1-py3.11 tensorflow_rt24.1-py3.11-dist tensorflow_rt24.1-py3.11-edt tensorflow_rt24.1-py3.11
XGBoost	1.6	xgboost_1.6 or scikit-learn_1.1 (see notes)	runtime-23.1-py3.10
XGBoost	2.0	xgboost_2.0 or scikit-learn_1.3	runtime-24.1-py3.11
onnx or onnxruntime	1.16	onnxruntime_1.16	onnxruntime_opset_19

IBM Watson Machine Learning Workflow



Evidently

Evidently

<https://evidentlyai.com/>

Product ▾

Pricing

Docs

Resources ▾

GitHub 5,693

Sign up

Get demo

Collaborative AI observability platform

Evaluate, test, and monitor your AI-powered products.

[Get started](#)

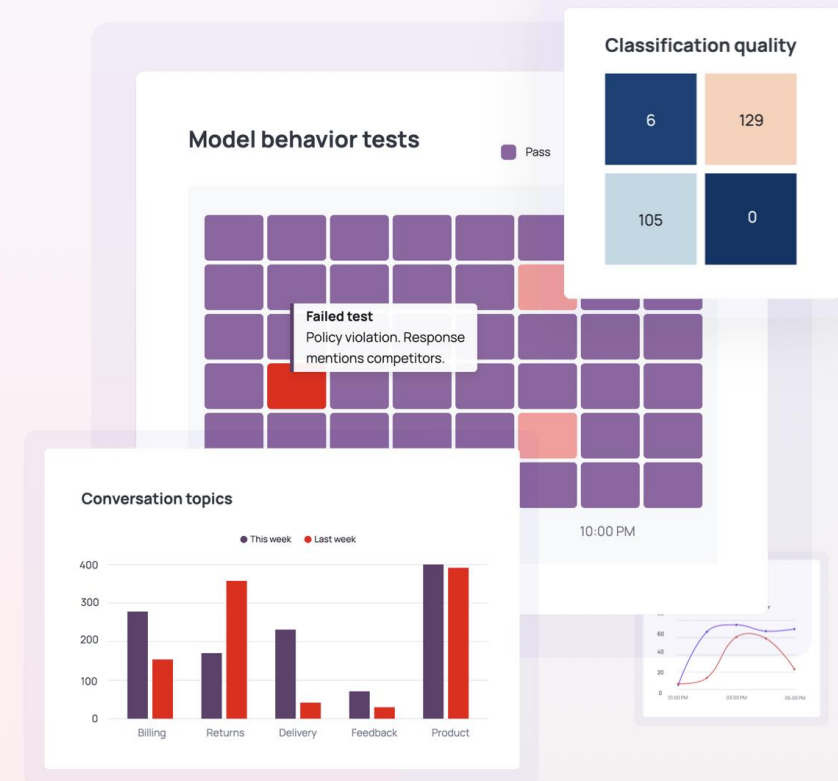
LLM and RAGs



ML models



Data pipelines

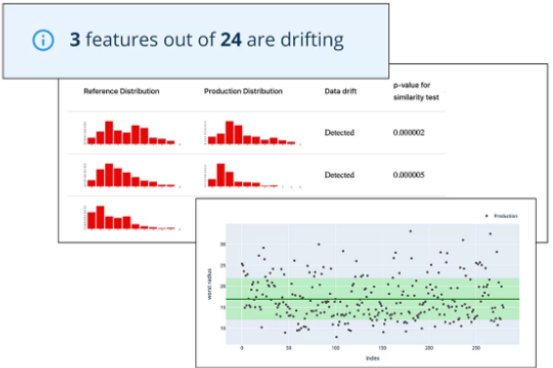
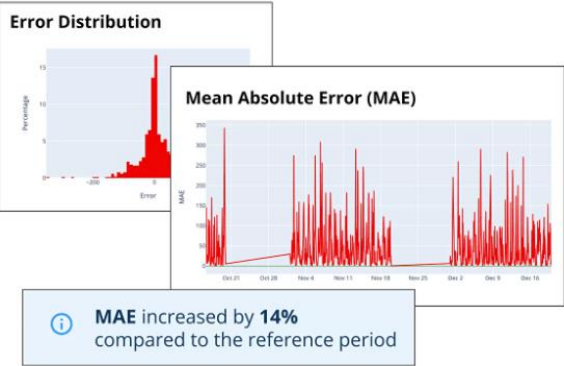


Evidently Model Monitoring

Model Quality

Evaluate the ML model quality, and go beyond aggregate performance to discover where it fails.

GET STARTED



Data Drift

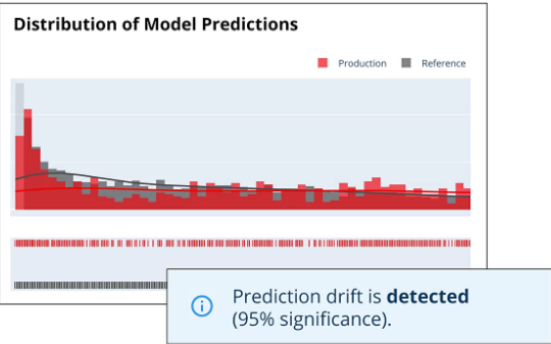
Run statistical tests to compare the input feature distributions, and visually explore the drift.

GET STARTED

Target Drift

Understand how model predictions and target behavior change over time.

GET STARTED



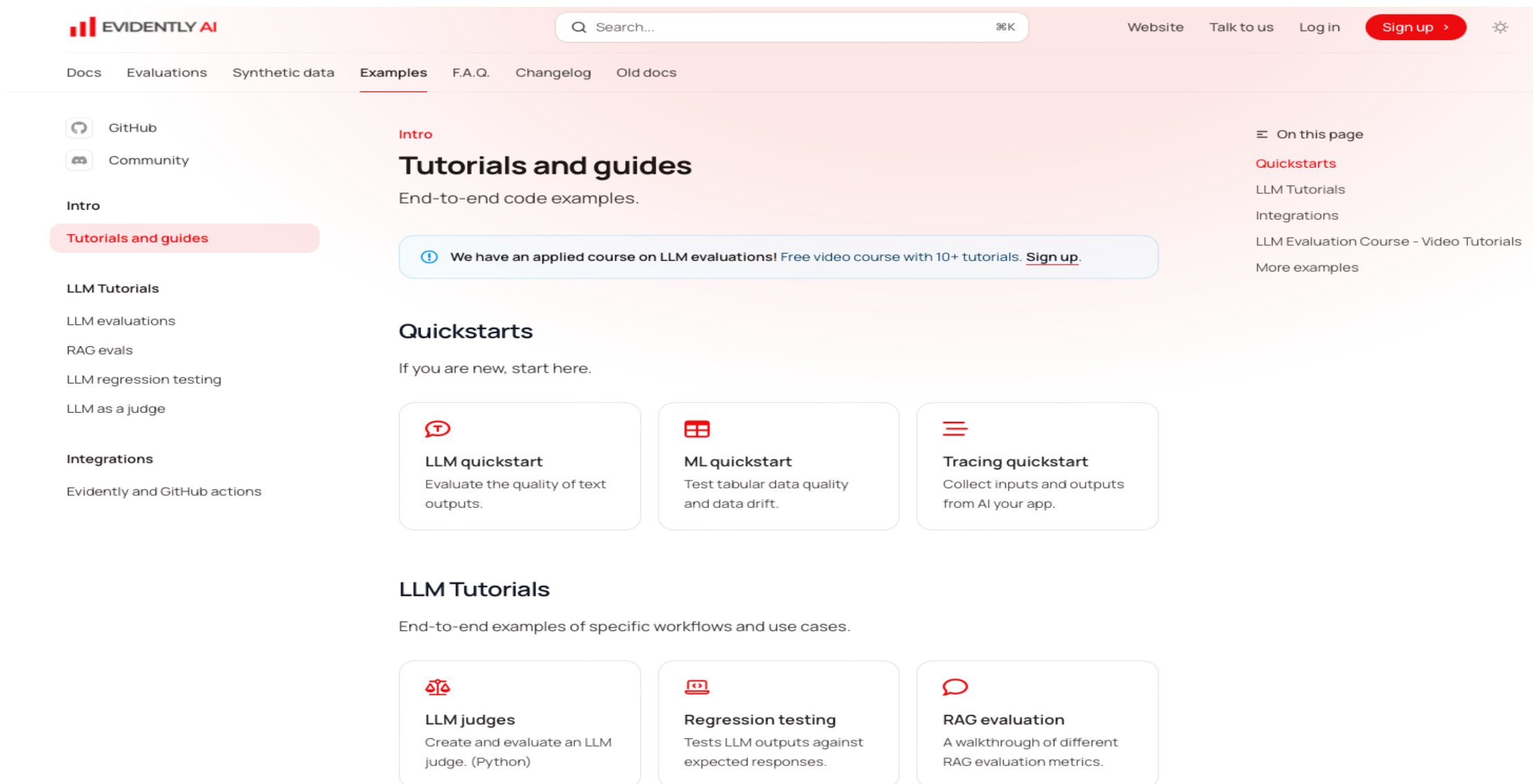
Data Quality

Get a snapshot of data health, and drill down to explore feature behavior and statistical properties.

GET STARTED

Evidently Sample Notebooks

<https://docs.evidentlyai.com/examples/introduction>



The screenshot shows the Evidently AI documentation website. The header includes the Evidently AI logo, a search bar, and navigation links for Website, Talk to us, Log in, and Sign up. The main navigation bar lists Docs, Evaluations, Synthetic data, Examples (highlighted), F.A.Q., Changelog, and Old docs. The left sidebar contains links to GitHub, Community, Intro, and a highlighted 'Tutorials and guides' section. Below this, there are sections for LLM Tutorials (LLM evaluations, RAG evals, LLM regression testing, LLM as a judge) and Integrations (Evidently and GitHub actions). The main content area is titled 'Tutorials and guides' with a sub-header 'Intro' and the text 'End-to-end code examples.' A blue banner promotes an applied course on LLM evaluations. Below this is the 'Quickstarts' section, which includes three cards: 'LLM quickstart' (Evaluate the quality of text outputs), 'ML quickstart' (Test tabular data quality and data drift), and 'Tracing quickstart' (Collect inputs and outputs from AI your app). The 'LLM Tutorials' section follows, with the text 'End-to-end examples of specific workflows and use cases.' and three cards: 'LLM judges' (Create and evaluate an LLM judge. (Python)), 'Regression testing' (Tests LLM outputs against expected responses), and 'RAG evaluation' (A walkthrough of different RAG evaluation metrics). The right sidebar, titled 'On this page', lists 'Quickstarts', 'LLM Tutorials', 'Integrations', 'LLM Evaluation Course - Video Tutorials', and 'More examples'.

EVIDENTLY AI Search... Website Talk to us Log in Sign up >

Docs Evaluations Synthetic data **Examples** F.A.Q. Changelog Old docs

GitHub
Community

Intro
Tutorials and guides

LLM Tutorials
LLM evaluations
RAG evals
LLM regression testing
LLM as a judge

Integrations
Evidently and GitHub actions

Intro
Tutorials and guides
End-to-end code examples.

We have an applied course on LLM evaluations! Free video course with 10+ tutorials. [Sign up.](#)

Quickstarts
If you are new, start here.

LLM quickstart
Evaluate the quality of text outputs.

ML quickstart
Test tabular data quality and data drift.

Tracing quickstart
Collect inputs and outputs from AI your app.

LLM Tutorials
End-to-end examples of specific workflows and use cases.

LLM judges
Create and evaluate an LLM judge. (Python)

Regression testing
Tests LLM outputs against expected responses.

RAG evaluation
A walkthrough of different RAG evaluation metrics.

On this page
Quickstarts
LLM Tutorials
Integrations
LLM Evaluation Course - Video Tutorials
More examples

IBM Watson OpenScale

IBM Watson OpenScale

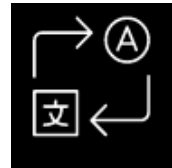
[Validating and monitoring AI models with Watson OpenScale](#)

Validate and monitor AI models, deployed anywhere, to help comply with regulations, address internal safeguards, and mitigate business risk



Monitoring for compliance and safeguards

- Mitigate biased model behavior
- Explain model decisions
- Validate and control risk



Ensure that models are resilient to changing situations

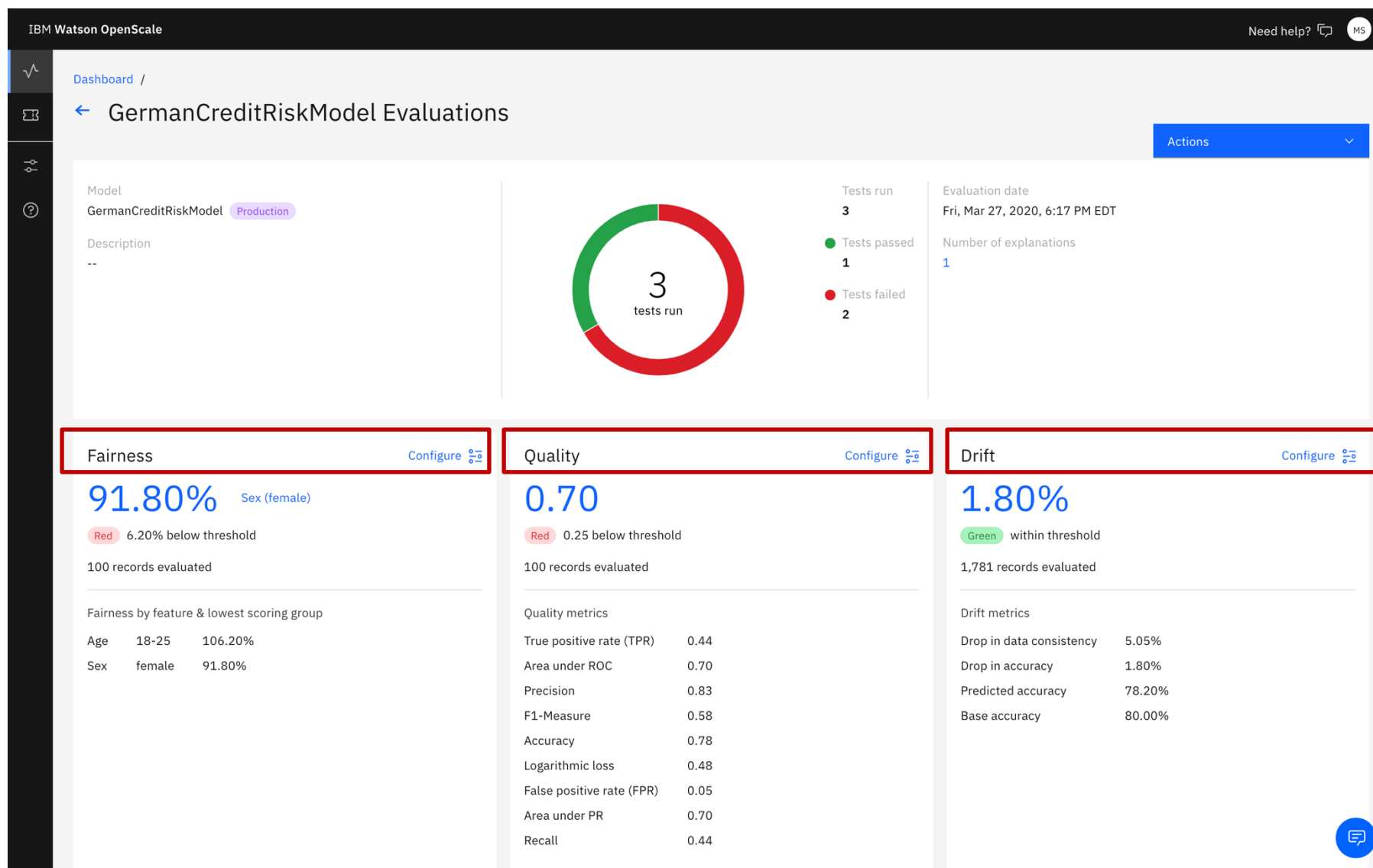
- Detect drift during runtime
- Generate specific model retraining inputs



Align model performance with business outcomes

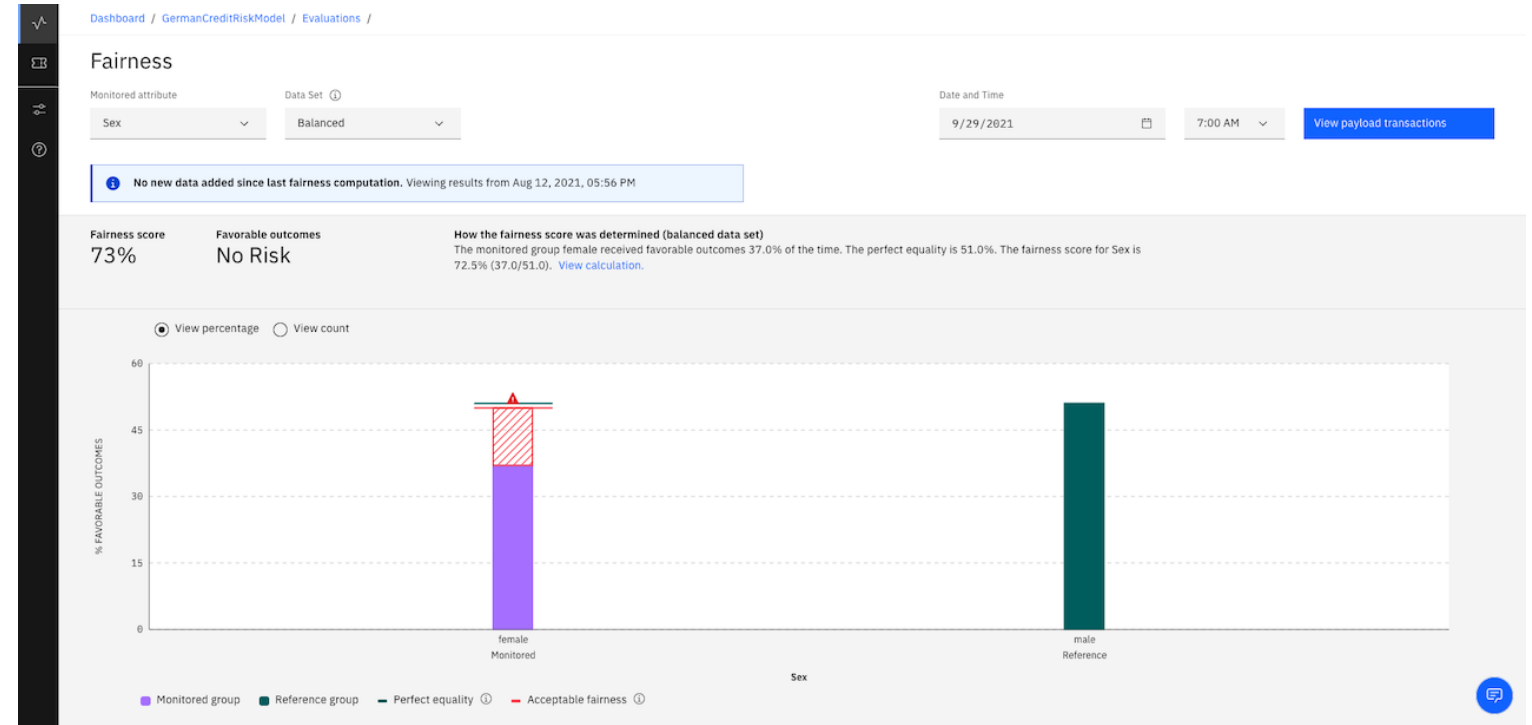
- Correlate model metrics and business KPIs
- Actionable metrics and alerts

Fairness, Quality and Drift Monitoring



Fairness Monitoring

- OpenScale compares expected scores for a control group against a reference group (usually sex, age, ethnic, etc.)



- Calculates **separation from expected value**
- Can generate debiased score in 'real time' for compliance (**OpenScale API can generate debiased score**)

Drift Monitoring

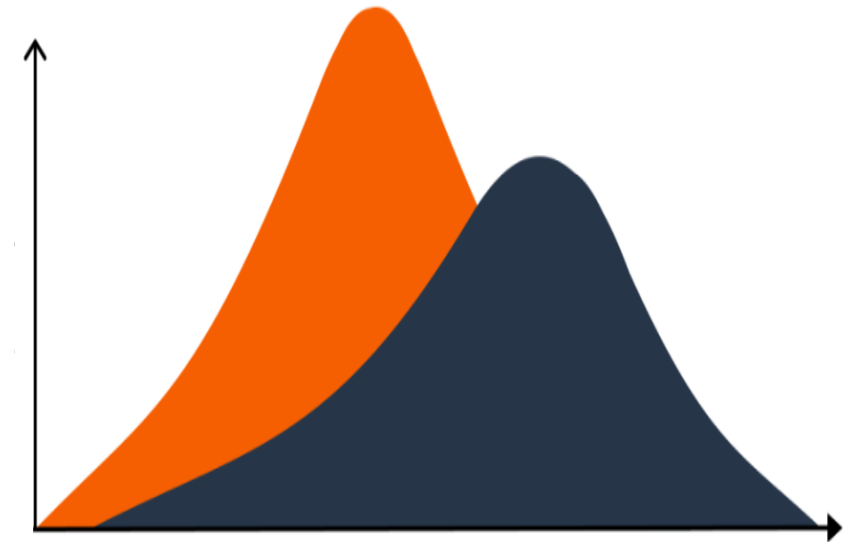
OpenScale monitors two types of data drift:

Drop in accuracy: Model accuracy could deviate if the data distribution changes in cases similar to the cases that the model was mistaken in training

Drop in data consistency: a drift in data happens when the distribution of cases changes from the training dataset to the cases that the model is scoring

Both metrics could raise an alert that the model should be retrained

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN



Model Quality

OpenScale monitors the most usual model quality metrics. This requires **feedback data**

Binary Classification

- Area under ROC
- True positive rate (TPR)
- Precision
- F1-Measure
- Logarithmic loss
- False positive rate (FPR)
- Area under PR
- Recall

Regression

- R squared
- Proportion explained variance
- Root of mean squared error
- Mean absolute error
- Mean squared error

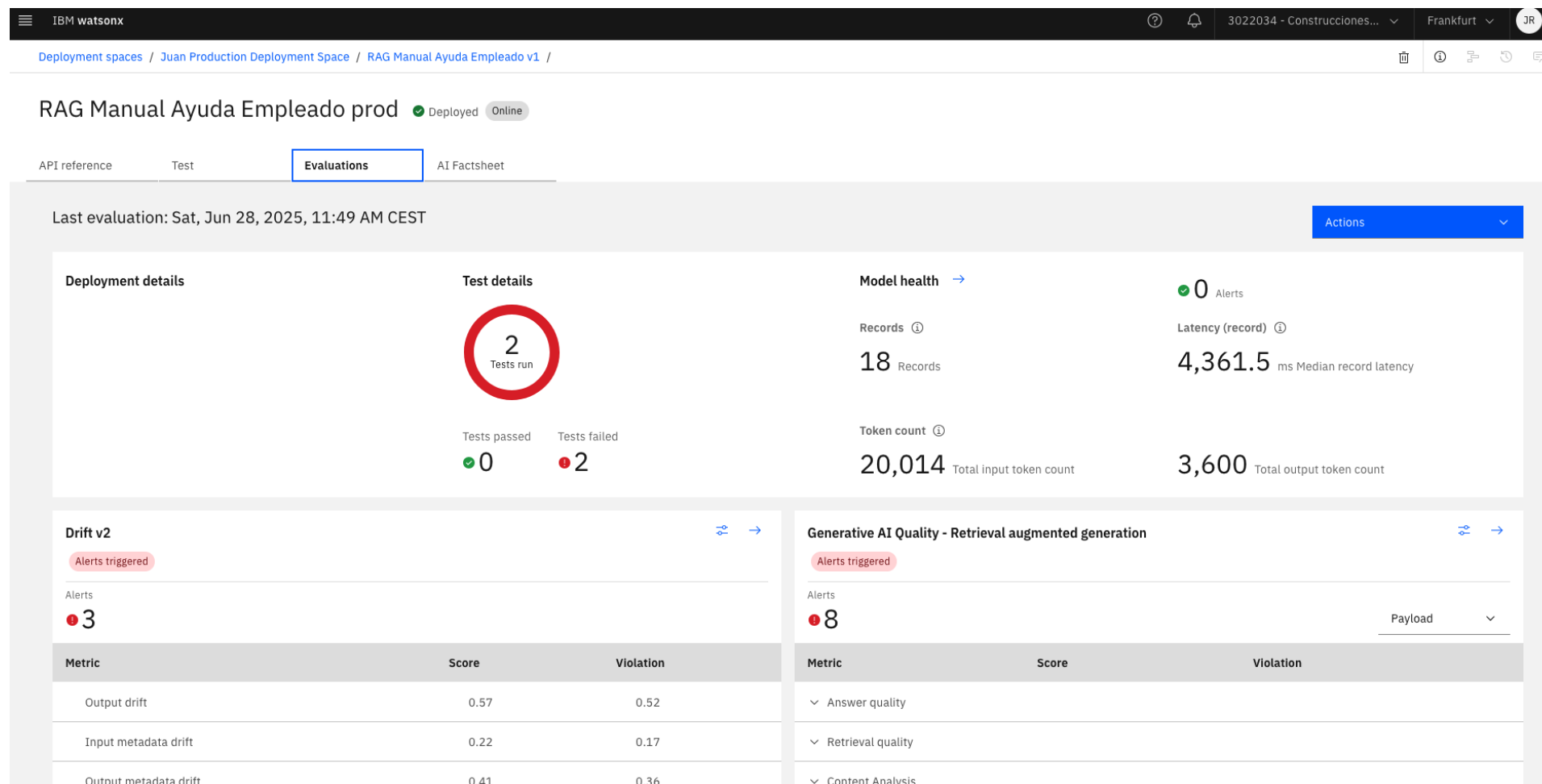
Multiclass Classification

- Accuracy
- Weighted True Positive Rate (wTPR)
- Weighted False Positive Rate (wFPR)
- Weighted recall
- Weighted precision
- Weighted F1-Measure
- Logarithmic loss

It is possible to create custom metrics and generate a custom monitor

Generative AI Monitoring

Different metrics depending on the use case: Classification, Entity Extraction, Generation, RAG, etc.



Example: Generative AI Quality in RAG Use Case

- For Payload data there are a collection of metrics that don't need Ground Truth
- Several metrics blocks:
 - Answer quality
 - Content analysis
 - Retrieval quality
 - Data safety



Model Health

- Monitor and understand model deployment usage, traffic, responsiveness and efficiency

